

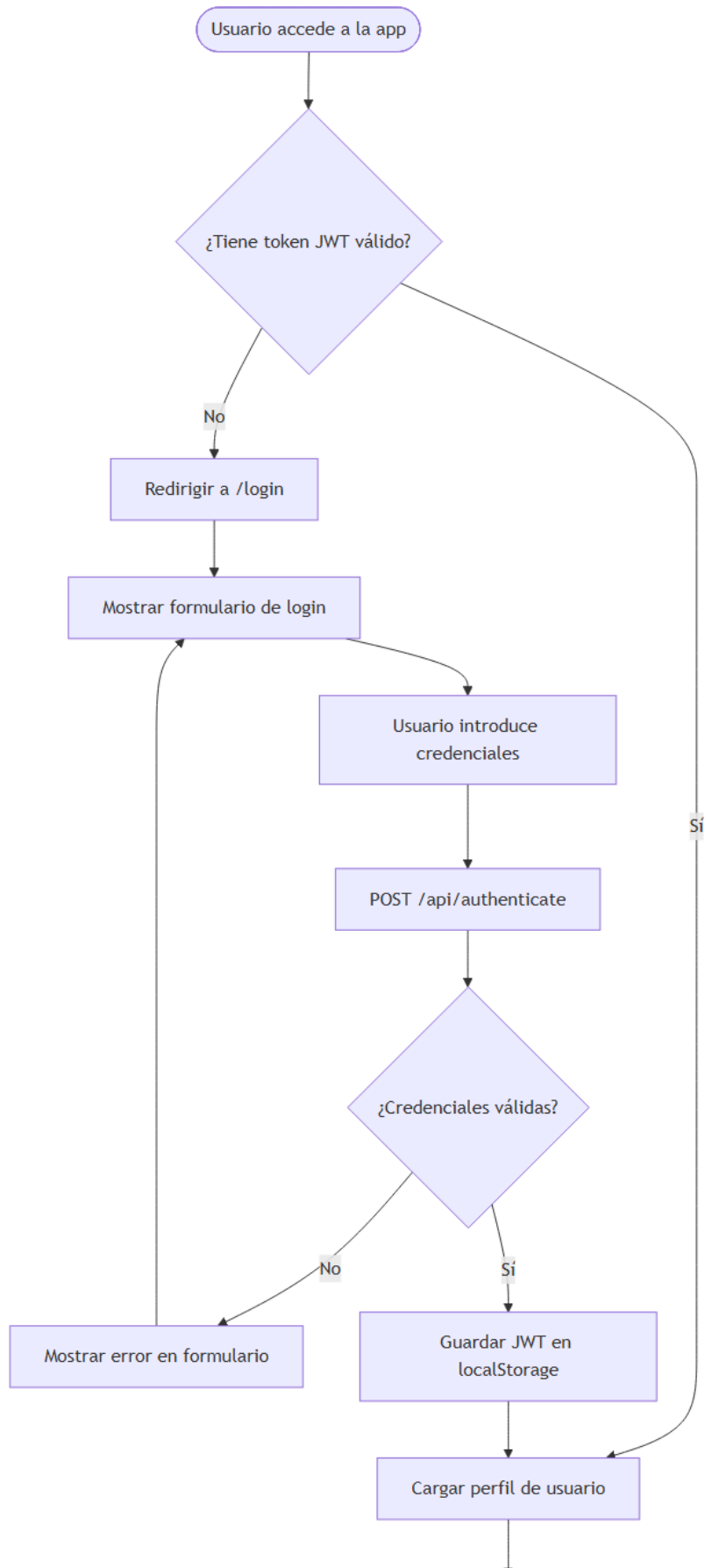
💡 Edita o prueba cualquier diagrama en mermaid.live — pega el código Mermaid del bloque correspondiente.

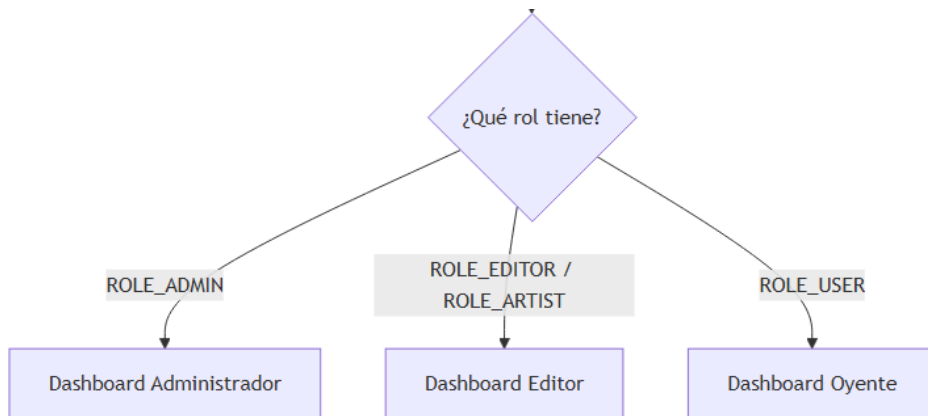
Diagramas de Flujo — MusicPlayer

Los diagramas están escritos en sintaxis **Mermaid** y se renderizan automáticamente en GitHub, GitLab y VS Code (extensión Mermaid Preview).

1. Flujo de autenticación

```
flowchart TD
    A([Usuario accede a la app]) --> B{¿Tiene token JWT válido?}
    B -- Sí --> C[Cargar perfil de usuario]
    B -- No --> D[Redirigir a /login]
    D --> E[Mostrar formulario de login]
    E --> F[Usuario introduce credenciales]
    F --> G[POST /api/authenticate]
    G --> H{¿Credenciales válidas?}
    H -- No --> I[Mostrar error en formulario]
    I --> E
    H -- Sí --> J[Guardar JWT en localStorage]
    J --> C
    C --> K{¿Qué rol tiene?}
    K -- ROLE_ADMIN --> L[Dashboard Administrador]
    K -- ROLE_EDITOR / ROLE_ARTIST --> M[Dashboard Editor]
    K -- ROLE_USER --> N[Dashboard Oyente]
```

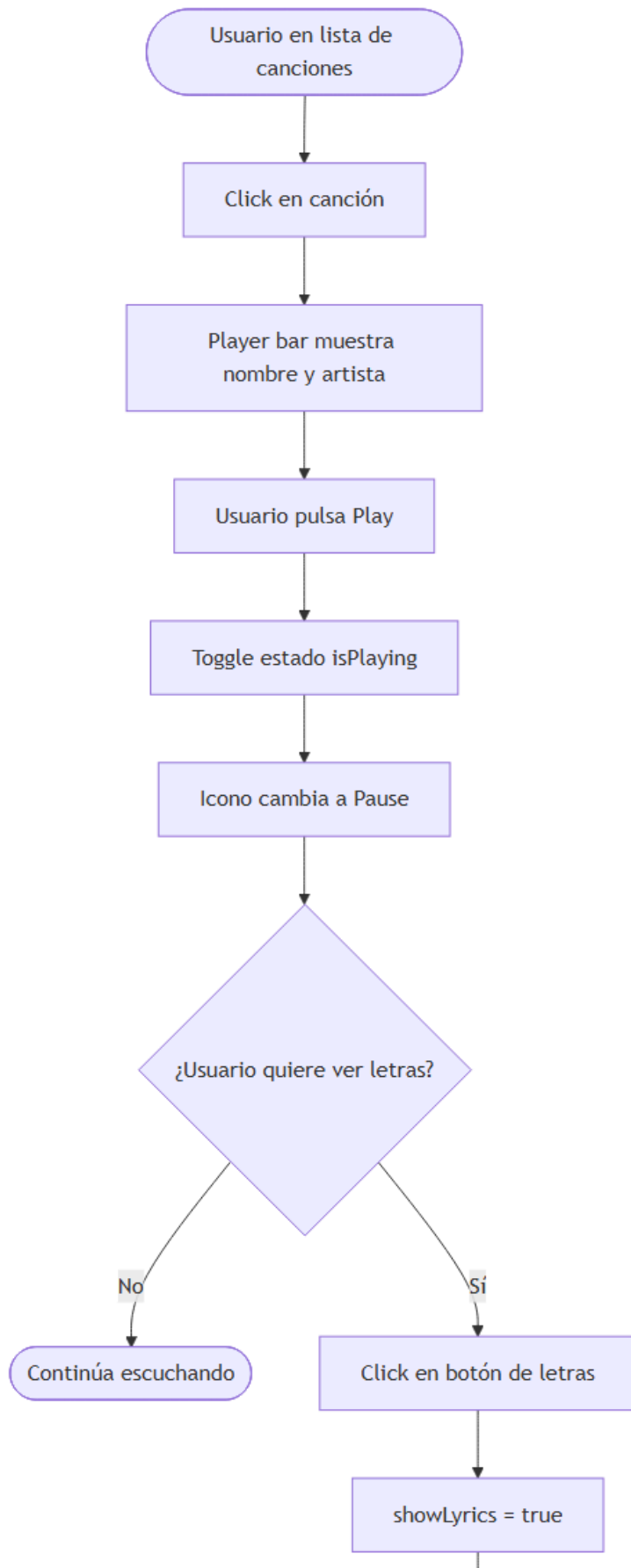


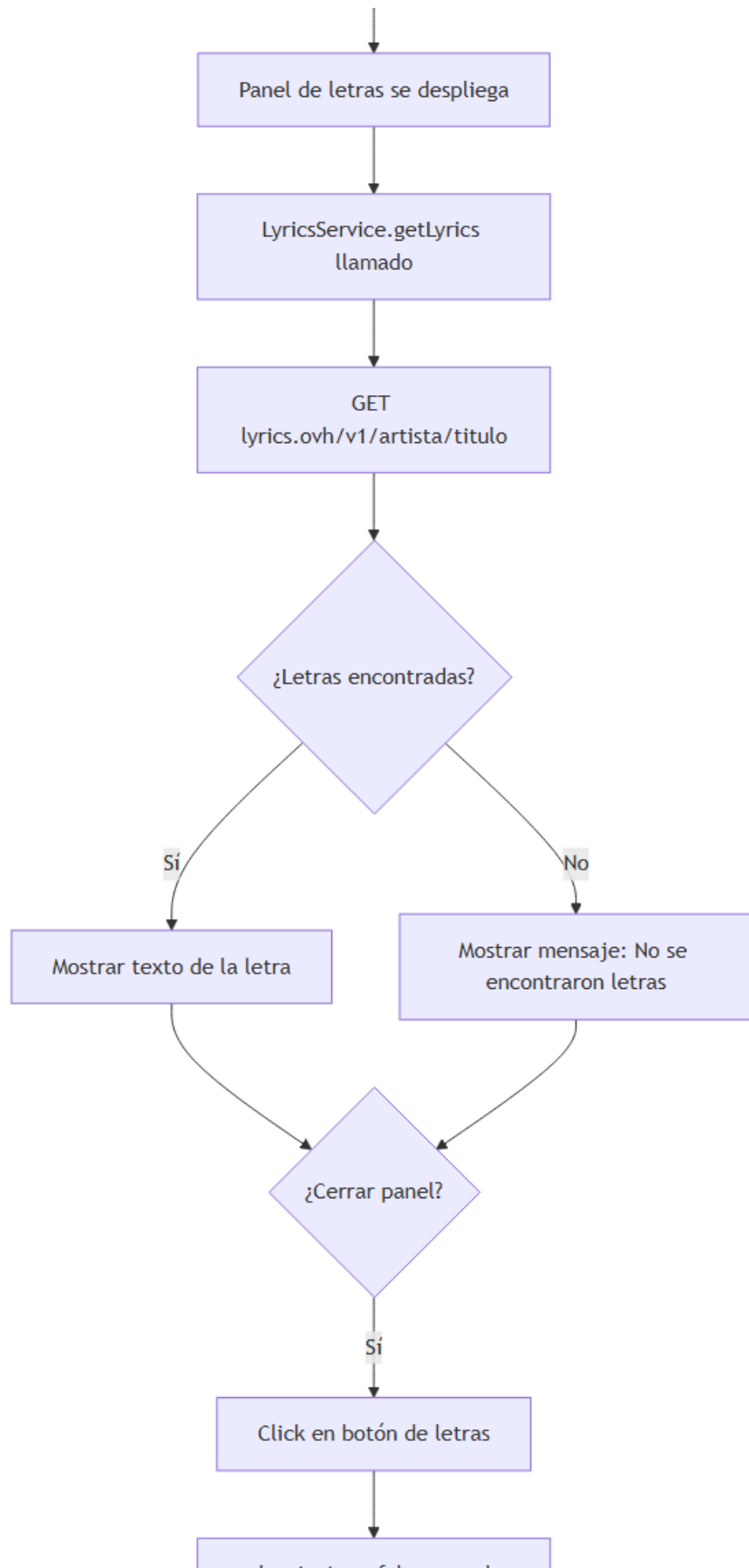


2. Flujo de reproducción y letras de canción

flowchart TD

```
A([Usuario en lista de canciones]) --> B[Click en canción]
B --> C[Player bar muestra nombre y artista]
C --> D[Usuario pulsa Play]
D --> E[Toggle estado isPlaying]
E --> F[Icono cambia a Pause]
F --> G{¿Usuario quiere ver letras?}
G -- No --> H[Continúa escuchando]
G -- Sí --> I[Click en botón de letras]
I --> J[showLyrics = true]
J --> K[Panel de letras se despliega]
K --> L[LyricsService.getLyrics llamado]
L --> M[GET lyrics.ovh/v1/artista/titulo]
M --> N{¿Letras encontradas?}
N -- Sí --> O[Mostrar texto de la letra]
N -- No --> P[Mostrar mensaje: No se encontraron letras]
O --> Q{¿Cerrar panel?}
P --> Q
Q -- Sí --> R[Click en botón de letras]
Q -- No --> S[showLyrics = false, panel se cierra]
```



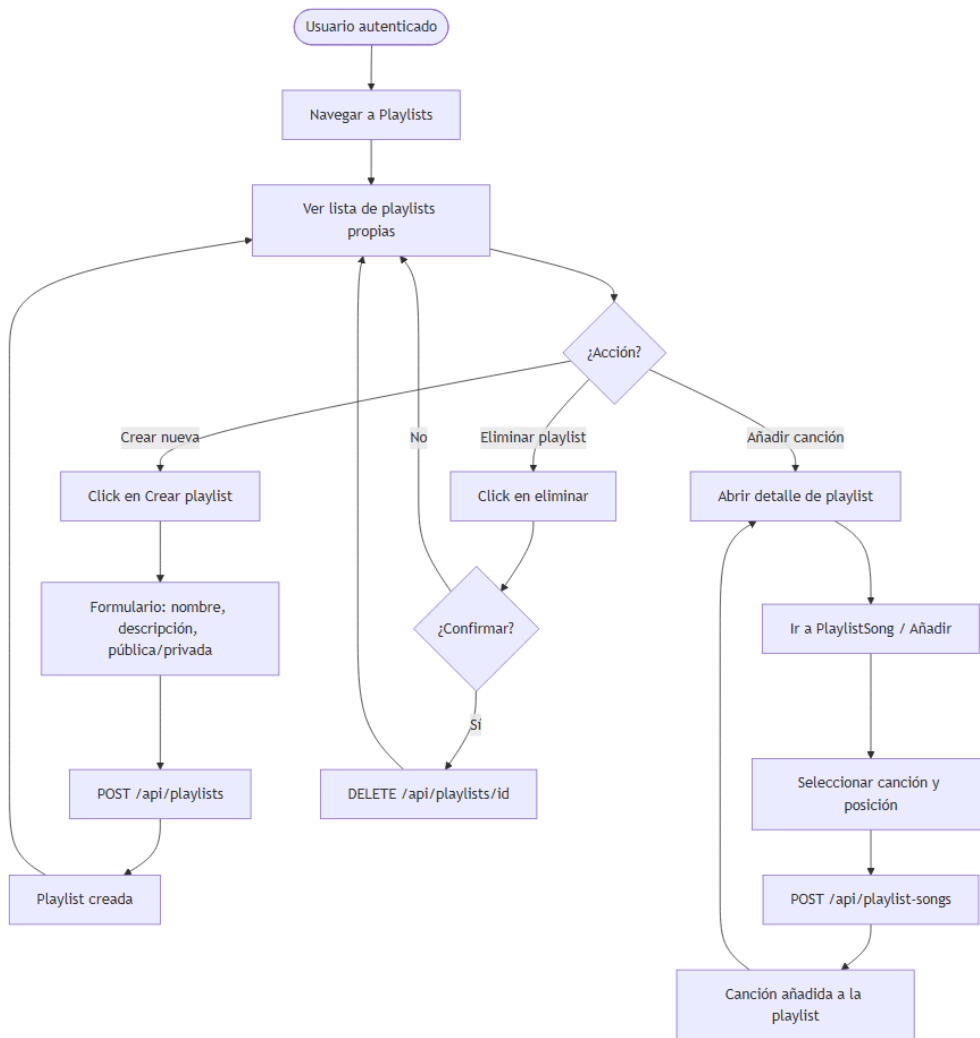


showLyrics = false, panel
se cierra

3. Flujo de gestión de playlist

flowchart TD

```
A([Usuario autenticado]) --> B[Navegar a Playlists]
B --> C[Ver lista de playlists propias]
C --> D{¿Acción?}
D -- Crear nueva --> E[Click en Crear playlist]
E --> F[Formulario: nombre, descripción, pública/privada]
F --> G[POST /api/playlists]
G --> H[Playlist creada]
H --> C
D -- Añadir canción --> I[Abrir detalle de playlist]
I --> J[Ir a PlaylistSong / Añadir]
J --> K[Seleccionar canción y posición]
K --> L[POST /api/playlist-songs]
L --> M[Canción añadida a la playlist]
M --> I
D -- Eliminar playlist --> N[Click en eliminar]
N --> O{¿Confirmar?}
O -- No --> C
O -- Sí --> P[DELETE /api/playlists/id]
P --> C
```

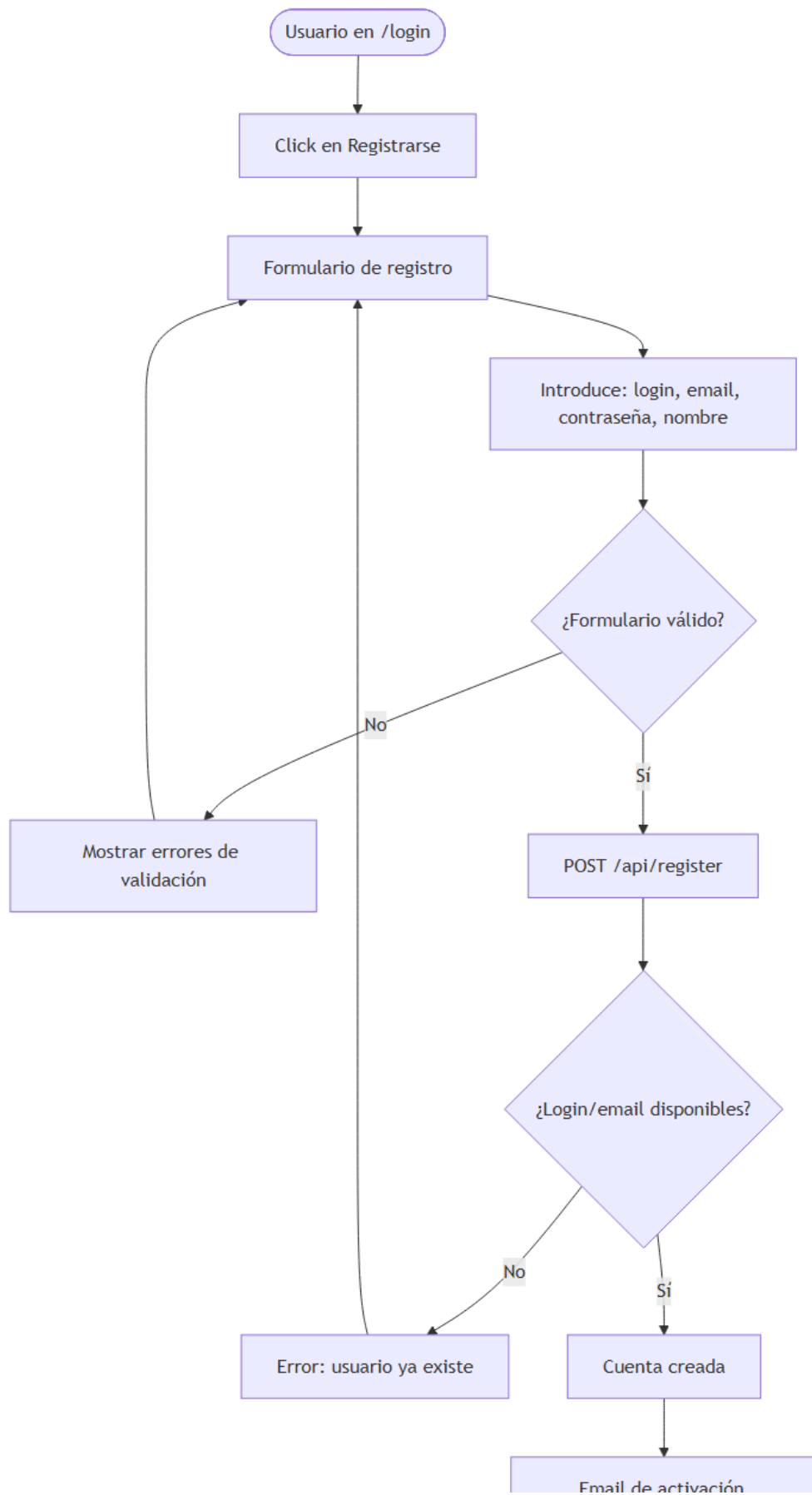


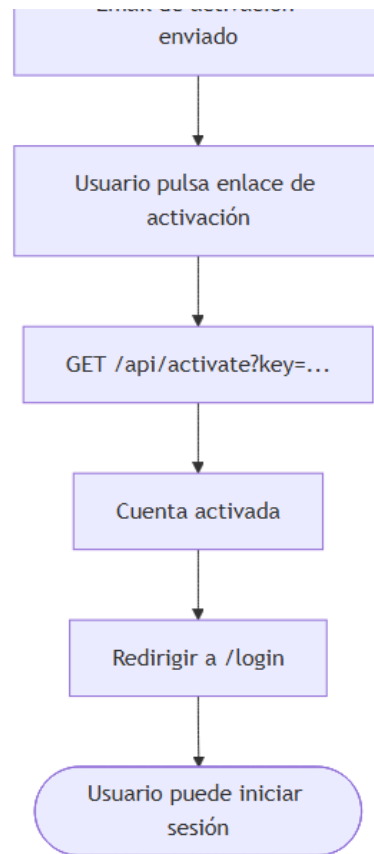
4. Flujo de registro de usuario

flowchart TD

```

A([Usuario en /login]) --> B[Click en Registrarse]
B --> C[Formulario de registro]
C --> D[Introduce: login, email, contraseña, nombre]
D --> E{¿Formulario válido?}
E -- No --> F[Mostrar errores de validación]
F --> C
E -- Sí --> G[POST /api/register]
G --> H{¿Login/email disponibles?}
H -- No --> I[Error: usuario ya existe]
I --> C
H -- Sí --> J[Cuenta creada]
J --> K[Email de activación enviado]
K --> L[Usuario pulsa enlace de activación]
L --> M[GET /api/activate?key=...]
M --> N[Cuenta activada]
N --> O[Redirigir a /login]
O --> P([Usuario puede iniciar sesión])
  
```





5. Diagrama de componentes Angular

```
graph TD
  subgraph AppRoot
    APP[App Component]
    MAIN[Main Component]
  end

  subgraph Layouts
    NAV[Navbar]
    SIDE[Sidebar]
    PLAYER[PlayerBar]
  end

  subgraph Home
    H[HomeComponent]
    DA[DashboardAdmin]
    DE[DashboardEditor]
    DU[DashboardUser]
  end

  subgraph Entities
    SONG[SongComponent]
    ALBUM[AlbumComponent]
    ARTIST[ArtistComponent]
    PLAYLIST[PlaylistComponent]
    GENRE[GenreComponent]
    LIKE[LikeComponent]
    PLAY[PlayComponent]
  end

  subgraph Core
    AUTH[AuthService]
  end
```



```

-- Uso: ejecutar en MySQL Workbench → luego
--      Database > Reverse Engineer → ver EER Diagram
-- =====

DROP DATABASE IF EXISTS music_player_components;
CREATE DATABASE music_player_components CHARACTER SET utf8mb4;
USE music_player_components;

-- — AppRoot —————
CREATE TABLE app_component (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'AppComponent'
);

CREATE TABLE main_component (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'MainComponent',
  app_component_id INT NOT NULL,
  FOREIGN KEY (app_component_id) REFERENCES app_component(id)
);

-- — Core (sin dependencias hacia arriba) —————
CREATE TABLE account_service (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'AccountService'
);

CREATE TABLE auth_service (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'AuthService'
);

CREATE TABLE interceptores_http (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'InterceptoresHTTP',
  auth_service_id INT NOT NULL,
  FOREIGN KEY (auth_service_id) REFERENCES auth_service(id)
);

-- — Layouts —————
CREATE TABLE navbar (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'Navbar',
  main_component_id INT NOT NULL,
  account_service_id INT NOT NULL,
  FOREIGN KEY (main_component_id) REFERENCES main_component(id),
  FOREIGN KEY (account_service_id) REFERENCES account_service(id)
);

CREATE TABLE sidebar (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'Sidebar',
  main_component_id INT NOT NULL,
  account_service_id INT NOT NULL,
  FOREIGN KEY (main_component_id) REFERENCES main_component(id),
  FOREIGN KEY (account_service_id) REFERENCES account_service(id)
);

CREATE TABLE lyrics_service (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'LyricsService'
);

CREATE TABLE player_bar (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'PlayerBar',
  main_component_id INT NOT NULL,
  lyrics_service_id INT NOT NULL,
  FOREIGN KEY (main_component_id) REFERENCES main_component(id),

```

```

FOREIGN KEY (lyrics_service_id) REFERENCES lyrics_service(id)
);

-- — Home —————
CREATE TABLE home_component (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'HomeComponent',
  main_component_id INT NOT NULL,
  account_service_id INT NOT NULL,
  FOREIGN KEY (main_component_id) REFERENCES main_component(id),
  FOREIGN KEY (account_service_id) REFERENCES account_service(id)
);

CREATE TABLE dashboard_admin (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'DashboardAdmin',
  home_component_id INT NOT NULL,
  FOREIGN KEY (home_component_id) REFERENCES home_component(id)
);

CREATE TABLE dashboard_editor (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'DashboardEditor',
  home_component_id INT NOT NULL,
  FOREIGN KEY (home_component_id) REFERENCES home_component(id)
);

CREATE TABLE dashboard_user (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'DashboardUser',
  home_component_id INT NOT NULL,
  FOREIGN KEY (home_component_id) REFERENCES home_component(id)
);

-- — Entities —————
CREATE TABLE song_component (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'SongComponent',
  main_component_id INT NOT NULL,
  FOREIGN KEY (main_component_id) REFERENCES main_component(id)
);

CREATE TABLE album_component (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'AlbumComponent',
  main_component_id INT NOT NULL,
  FOREIGN KEY (main_component_id) REFERENCES main_component(id)
);

CREATE TABLE artist_component (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'ArtistComponent',
  main_component_id INT NOT NULL,
  FOREIGN KEY (main_component_id) REFERENCES main_component(id)
);

CREATE TABLE playlist_component (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'PlaylistComponent',
  main_component_id INT NOT NULL,
  FOREIGN KEY (main_component_id) REFERENCES main_component(id)
);

CREATE TABLE genre_component (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'GenreComponent',
  main_component_id INT NOT NULL,
  FOREIGN KEY (main_component_id) REFERENCES main_component(id)
);

```

```

CREATE TABLE like_component (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'LikeComponent',
  main_component_id INT NOT NULL,
  FOREIGN KEY (main_component_id) REFERENCES main_component(id)
);

CREATE TABLE play_component (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) DEFAULT 'PlayComponent',
  main_component_id INT NOT NULL,
  FOREIGN KEY (main_component_id) REFERENCES main_component(id)
);

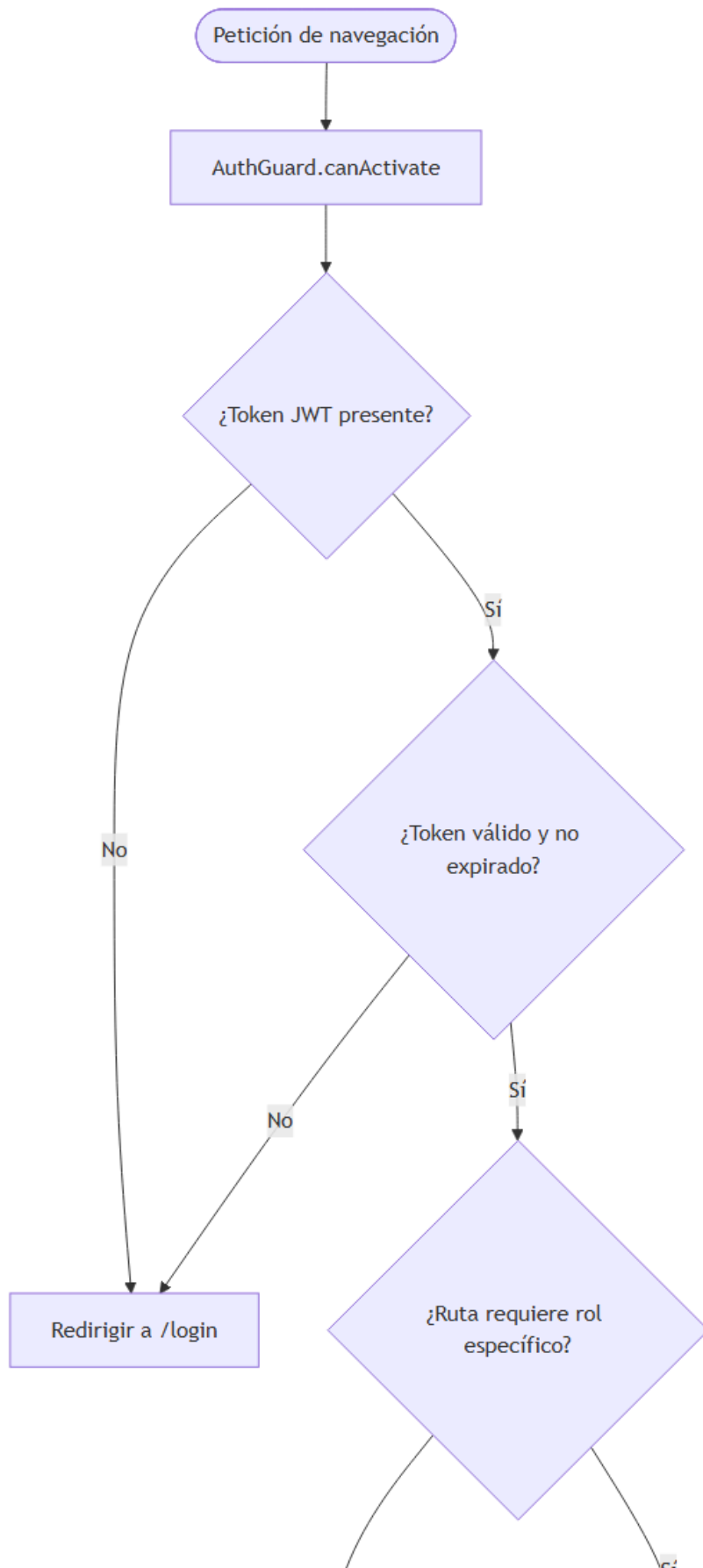
```

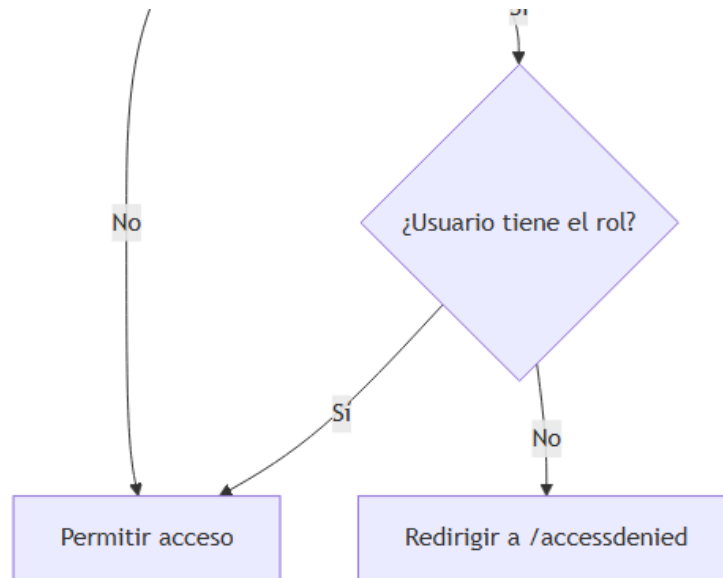
6. Flujo de control de acceso por rol

```

flowchart TD
  A([Petición de navegación]) --> B[AuthGuard.canActivate]
  B --> C{¿Token JWT presente?}
  C -- No --> D[Redirigir a /login]
  C -- Sí --> E{¿Token válido y no expirado?}
  E -- No --> D
  E -- Sí --> F{¿Ruta requiere rol específico?}
  F -- No --> G[Permitir acceso]
  F -- Sí --> H{¿Usuario tiene el rol?}
  H -- Sí --> G
  H -- No --> I[Redirigir a /accessdenied]

```





7. Flujo de subida de ficheros (imagen y audio)

flowchart TD

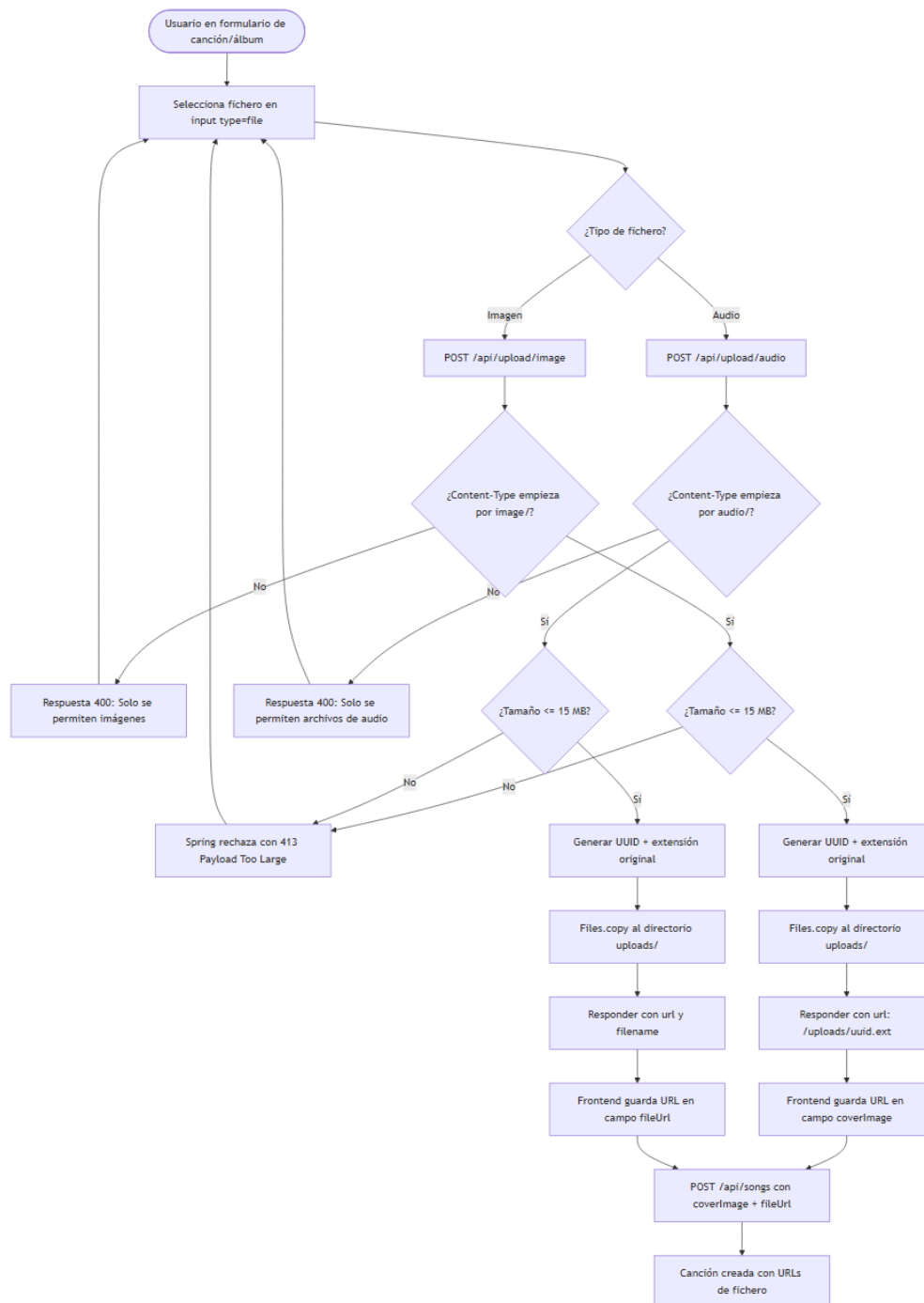
```

A([Usuario en formulario de canción/álbum]) --> B[Selecciona fichero en input type=file]
B --> C{¿Tipo de fichero?}
C -- Imagen --> D[POST /api/upload/image]
C -- Audio --> E[POST /api/upload/audio]

D --> F{¿Content-Type empieza por image/?}
F -- No --> G[Respuesta 400: Solo se permiten imágenes]
G --> B
F -- Sí --> H{¿Tamaño <= 15 MB?}
H -- No --> I[Spring rechaza con 413 Payload Too Large]
I --> B
H -- Sí --> J[Generar UUID + extensión original]
J --> K[Files.copy al directorio uploads/]
K --> L[Responder con url: /uploads/uuid.ext]
L --> M[Frontend guarda URL en campo coverImage]

E --> N{¿Content-Type empieza por audio/?}
N -- No --> O[Respuesta 400: Solo se permiten archivos de audio]
O --> B
N -- Sí --> P{¿Tamaño <= 15 MB?}
P -- No --> I
P -- Sí --> Q[Generar UUID + extensión original]
Q --> R[Files.copy al directorio uploads/]
R --> S[Responder con url y filename]
S --> T[Frontend guarda URL en campo fileUrl]

M --> U[POST /api/songs con coverImage + fileUrl]
T --> U
U --> V[Canción creada con URLs de fichero]
  
```



8. Flujo de streaming de audio con rangos HTTP

sequenceDiagram

participant C as Navegador (HTML5 audio)

participant F as Frontend Angular

participant B as Backend Spring Boot

participant D as Disco (uploads/)

F->>F: currentSong.fileUrl = /uploads/abc.mp3

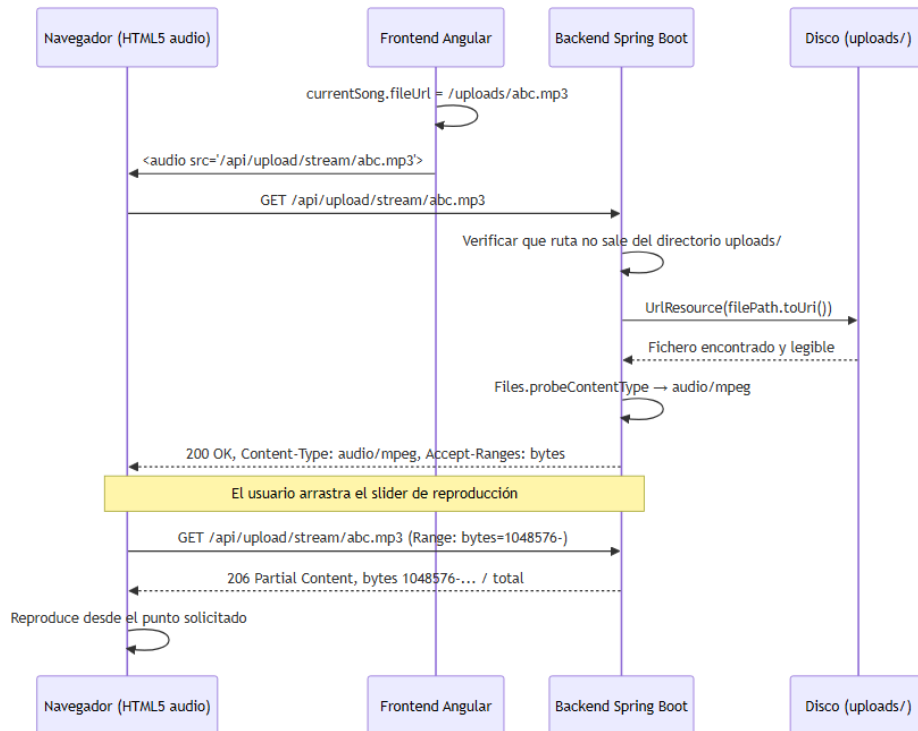
F->>C: <audio src="/api/upload/stream/abc.mp3">

C->>B: GET /api/upload/stream/abc.mp3

B->>B: Verificar que ruta no sale del directorio uploads/

B->>D: `UrlResource(filePath.toUri())`
D->>B: Fichero encontrado y legible
B->>B: `Files.probeContentType` → audio/mpeg
B->>C: 200 OK, Content-Type: audio/mpeg, Accept-Ranges: bytes

Note over C,B: El usuario arrastra el slider de reproducción
C->>B: `GET /api/upload/stream/abc.mp3 (Range: bytes=1048576-)`
B->>C: 206 Partial Content, bytes 1048576-... / total
C->>C: Reproduce desde el punto solicitado



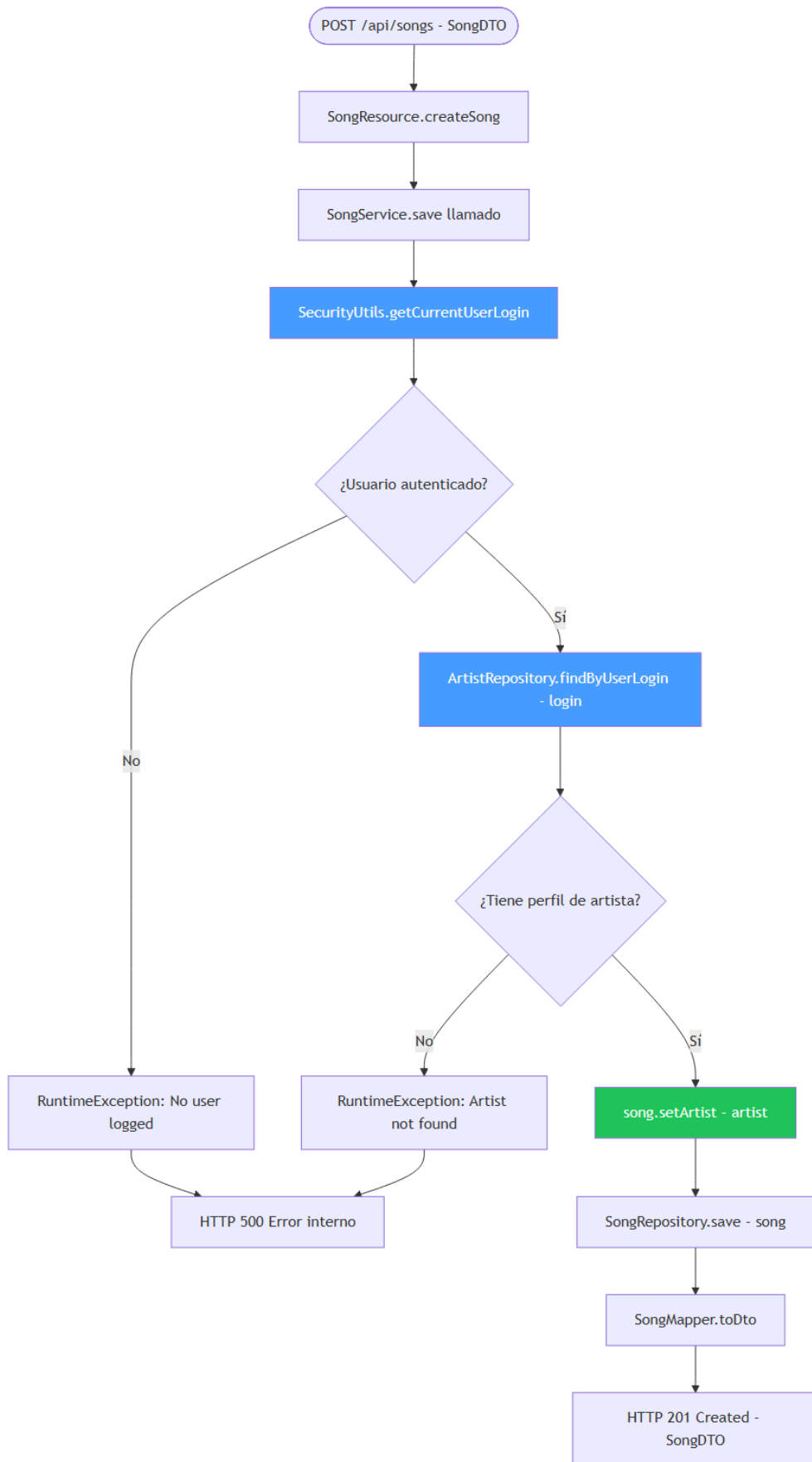
9. Patrón de propiedad en la capa de servicio (Ownership)

flowchart TD

```

A([POST /api/songs - SongDTO]) --> B[SongResource.createSong]
B --> C[SongService.save llamado]
C --> D[SecurityUtils.getCurrentUserLogin]
D --> E[{Usuario autenticado?}]
E -- No --> F[RuntimeException: No user logged]
F --> G[HTTP 500 Error interno]
E -- Sí --> H[ArtistRepository.findByUserLogin - login]
H --> I[{Tiene perfil de artista?}]
I -- No --> J[RuntimeException: Artist not found]
J --> G
I -- Sí --> K[song.setArtist - artist]
K --> L[SongRepository.save - song]
L --> M[SongMapper.toDto]
M --> N[HTTP 201 Created - SongDTO]
  
```

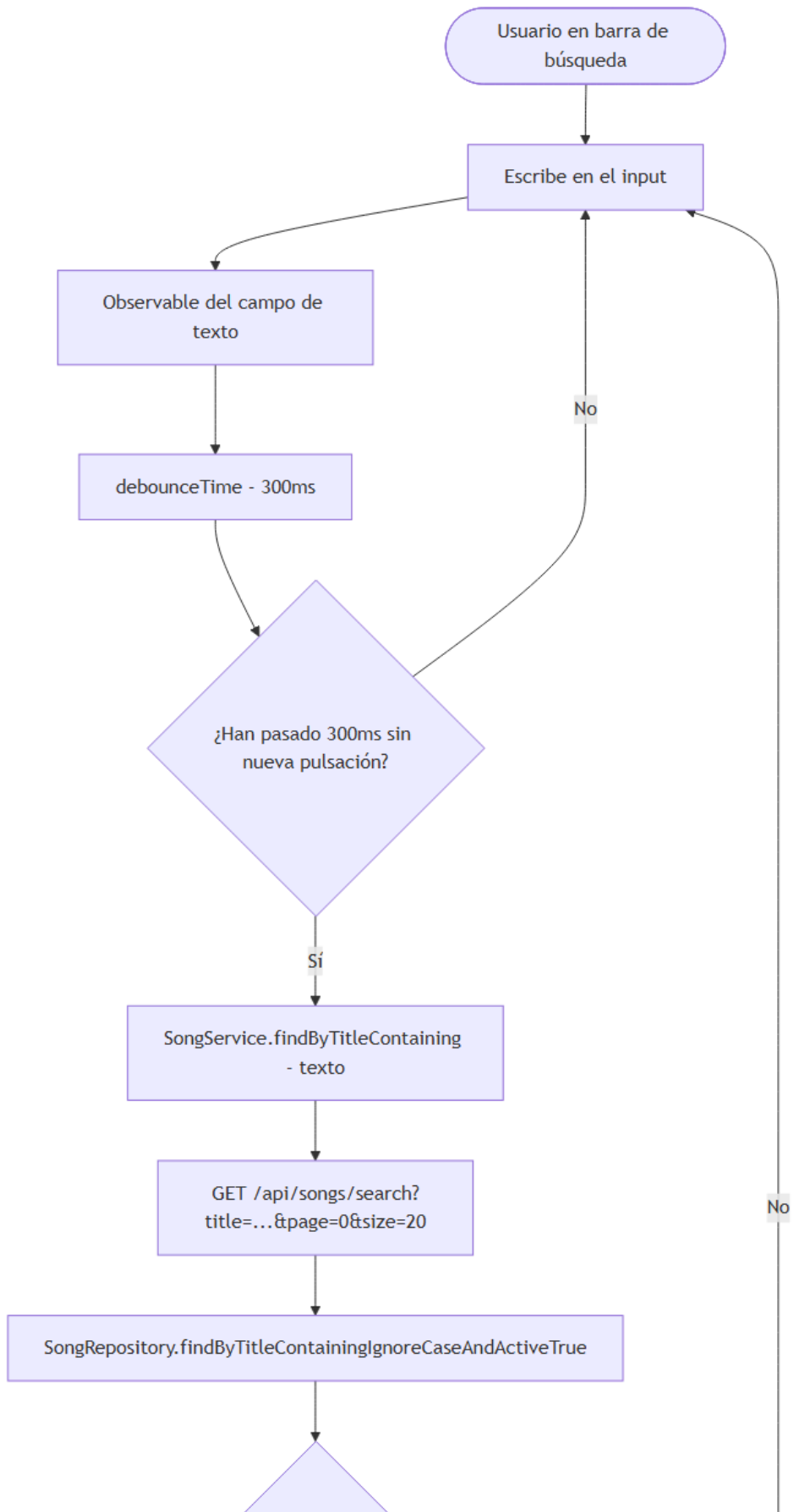
style D fill:#4a9eff,color:#fff
style H fill:#4a9eff,color:#fff
style K fill:#22c55e,color:#fff

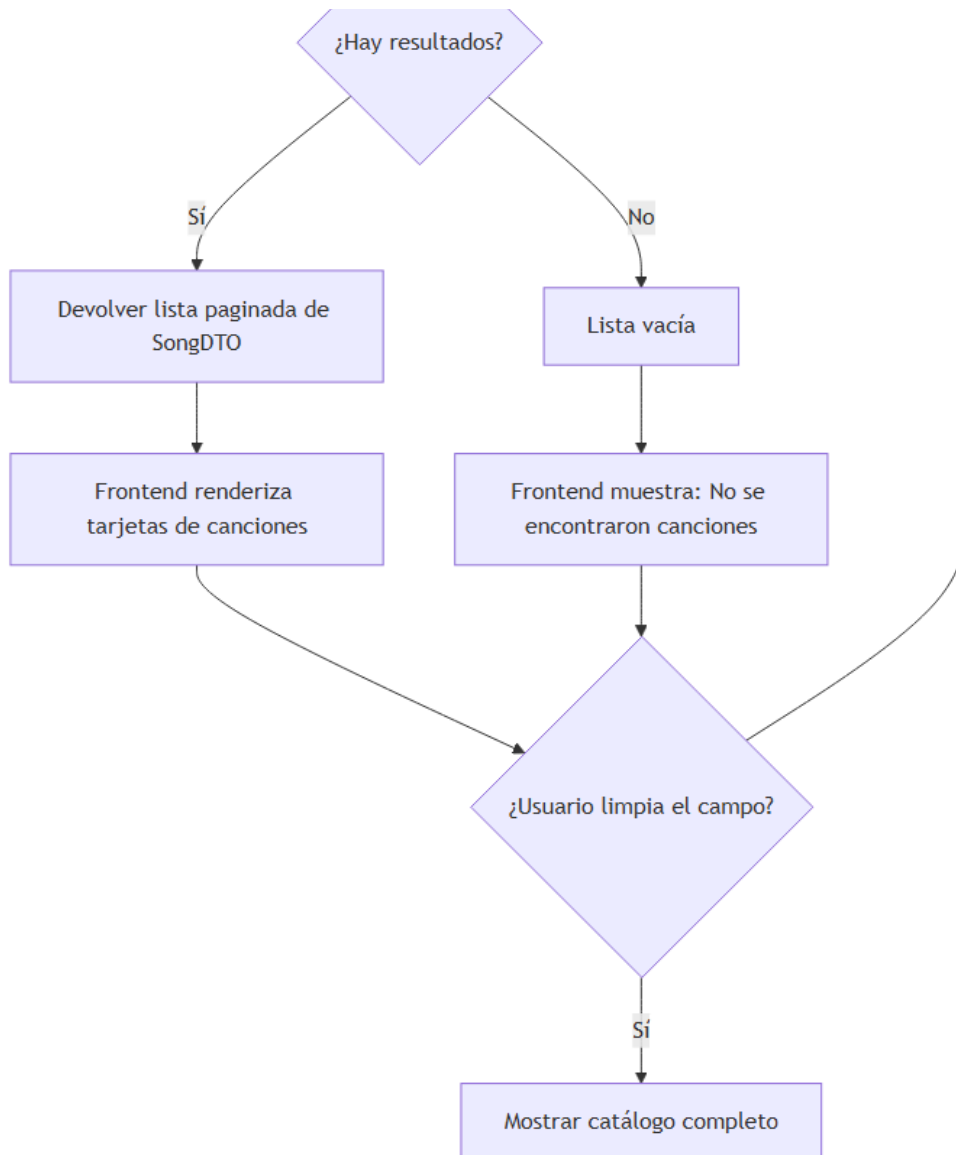


10. Flujo de búsqueda de canciones en tiempo real

flowchart TD

```
A([Usuario en barra de búsqueda]) --> B[Escribe en el input]
B --> C[Observable del campo de texto]
C --> D[debounceTime - 300ms]
D --> E{¿Han pasado 300ms sin nueva pulsación?}
E -- No --> B
E -- Sí --> F[SongService.findByTitleContaining - texto]
F --> G[GET /api/songs/search?title=...&page=0&size=20]
G --> H[SongRepository.findByTitleContainingIgnoreCaseAndActiveTrue]
H --> I{¿Hay resultados?}
I -- Sí --> J[Devolver lista paginada de SongDTO]
J --> K[Frontend renderiza tarjetas de canciones]
I -- No --> L[Lista vacía]
L --> M[Frontend muestra: No se encontraron canciones]
K --> N{¿Usuario limpia el campo?}
M --> N
N -- Sí --> O[Mostrar catálogo completo]
N -- No --> B
```



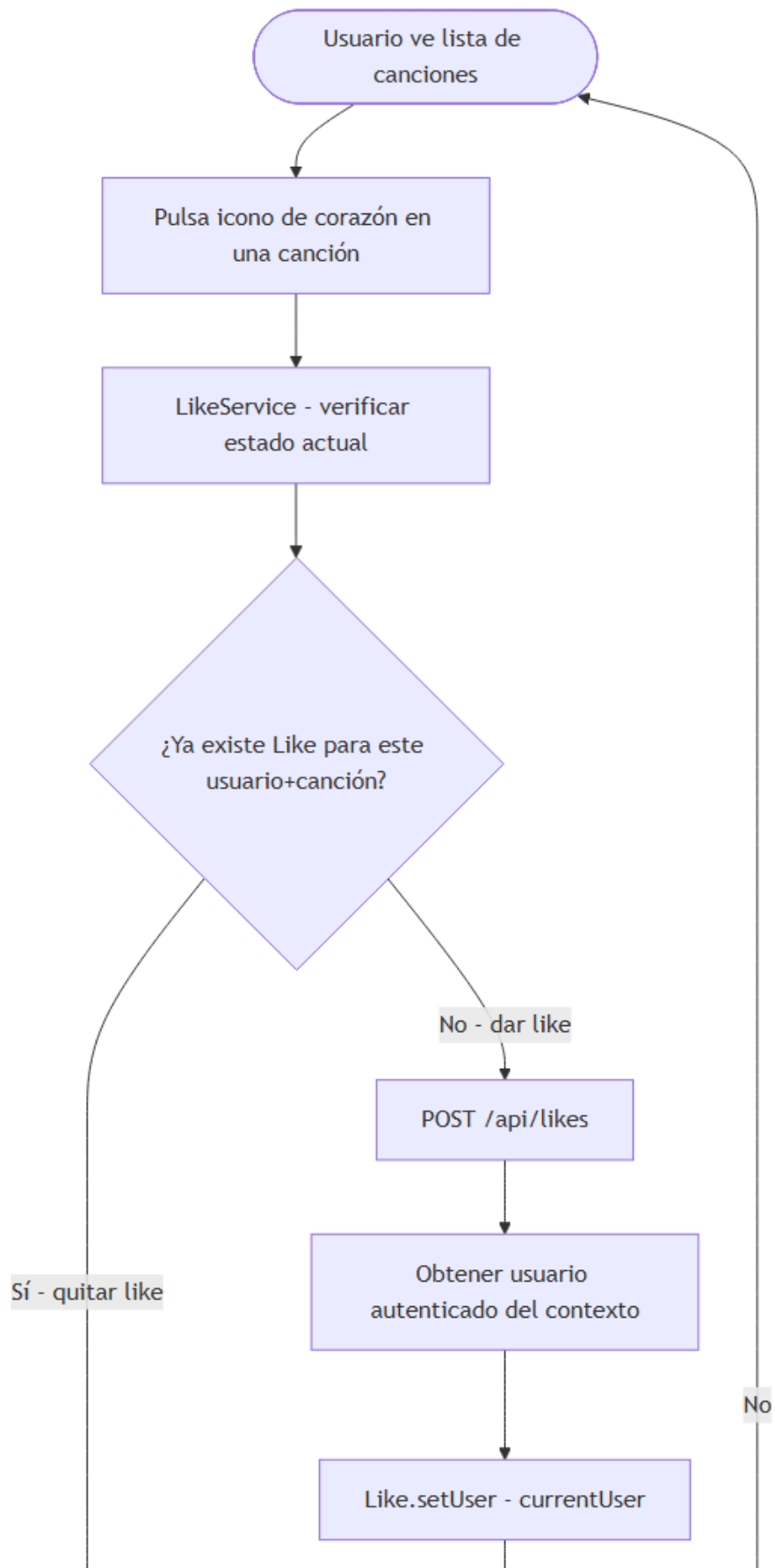


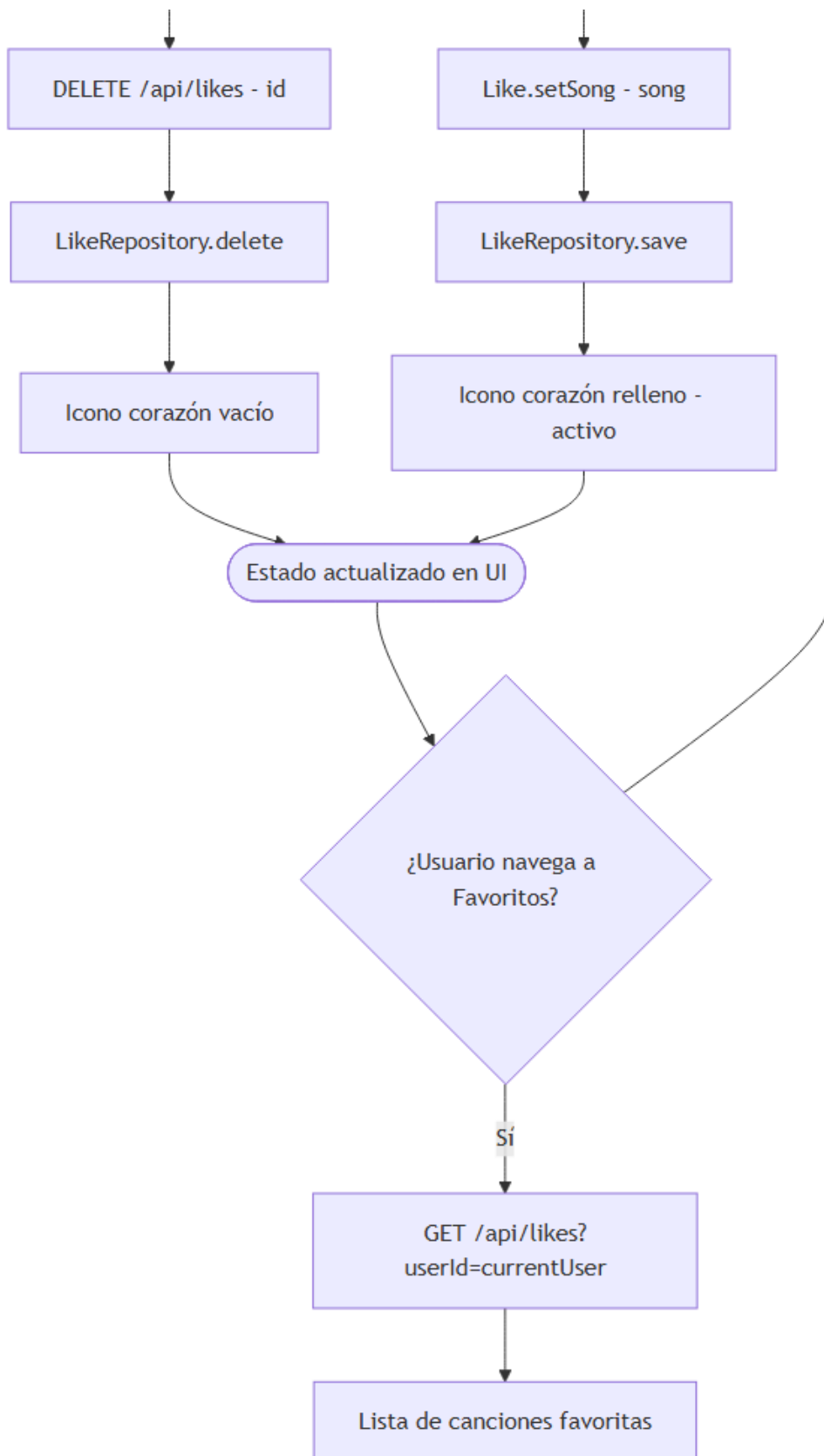
11. Flujo de canciones favoritas (Likes)

flowchart TD

```

A([Usuario ve lista de canciones]) --> B[Pulsa icono de corazón en una canción]
B --> C[LikeService - verificar estado actual]
C --> D{¿Ya existe Like para este usuario+canción?}
D -- Sí - quitar like --> E[DELETE /api/likes - id]
E --> F[LikeRepository.delete]
F --> G[Icono corazón vacío]
D -- No - dar like --> H[POST /api/likes]
H --> I[Obtener usuario autenticado del contexto]
I --> J[Like.setUser - currentUser]
J --> K[Like.setSong - song]
K --> L[LikeRepository.save]
L --> M[Icono corazón relleno - activo]
M --> N[Estado actualizado en UI]
N --> O{¿Usuario navega a Favoritos?}
O -- Sí --> P[GET /api/likes?userId=currentUser]
P --> Q[Lista de canciones favoritas]
O -- No --> A
  
```



12. Diagrama de secuencia — Ciclo completo de creación de canción

```
sequenceDiagram
    participant U as Usuario (Artista)
    participant A as Angular Frontend
```

participant S as Spring Boot Backend
participant DB as MySQL

U->>A: Navegar a /song/new
A->>A: Mostrar formulario de creación

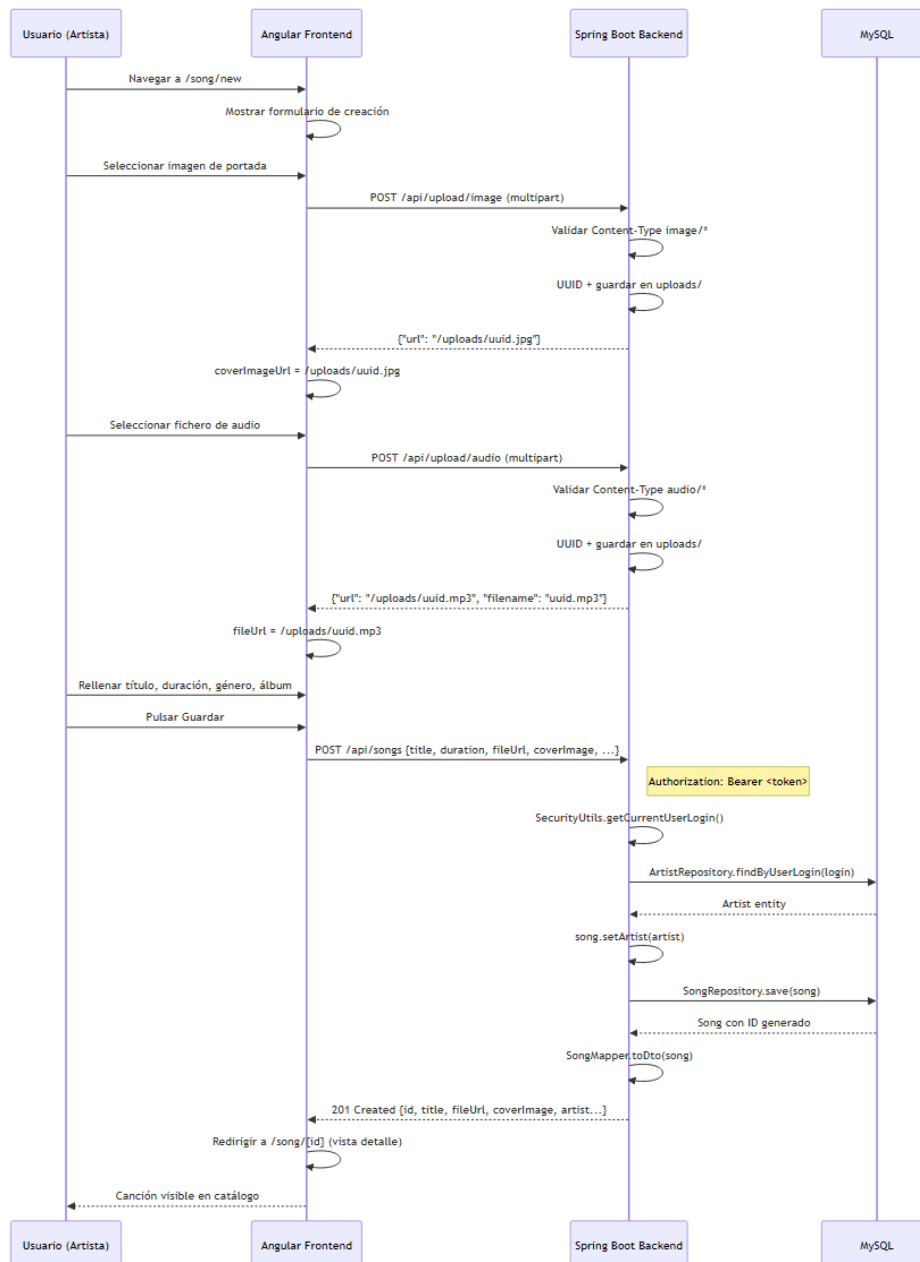
U->>A: Seleccionar imagen de portada
A->>S: POST /api/upload/image (multipart)
S->>S: Validar Content-Type image/*
S->>S: UUID + guardar en uploads/
S-->>A: {"url": "/uploads/uuid.jpg"}
A->>A: coverImageUrl = /uploads/uuid.jpg

U->>A: Seleccionar fichero de audio
A->>S: POST /api/upload/audio (multipart)
S->>S: Validar Content-Type audio/*
S->>S: UUID + guardar en uploads/
S-->>A: {"url": "/uploads/uuid.mp3", "filename": "uuid.mp3"}
A->>A: fileUrl = /uploads/uuid.mp3

U->>A: Rellenar título, duración, género, álbum
U->>A: Pulsar Guardar

A->>S: POST /api/songs {title, duration, fileUrl, coverImage, ...}
Note right of S: Authorization: Bearer <token>
S->>S: SecurityUtils.getCurrentUserLogin()
S->>DB: ArtistRepository.findByUserLogin(login)
DB-->>S: Artist entity
S->>S: song.setArtist(artist)
S->>DB: SongRepository.save(song)
DB-->>S: Song con ID generado
S->>S: SongMapper.toDto(song)
S-->>A: 201 Created {id, title, fileUrl, coverImage, artist...}

A->>A: Redirigir a /song/{id} (vista detalle)
A-->>U: Canción visible en catálogo



13. Diagrama de componentes Backend — Capa de servicio

```

graph TD
    subgraph "Controladores REST (web.rest)"
        SR[SongResource]
        AR[AlbumResource]
        ATR[ArtistResource]
        PR[PlaylistResource]
        FU[FileUploadResource]
        GR[GenreResource]
    end

    subgraph "Servicios (service.impl)"
        SS[SongServiceImpl]
        AS[AlbumServiceImpl]
        ATS[ArtistServiceImpl]
        PS[PlaylistServiceImpl]
    end
  
```

```

    LS[LikeService]
end

subgraph "Repositorios (repository)"
    SRepo[SongRepository]
    ARepo[AlbumRepository]
    ATRepo[ArtistRepository]
    PRepo[PlaylistRepository]
    URepo[UserRepository]
    LRepo[LikeRepository]
end

subgraph "Seguridad"
    SEC[SecurityUtils]
    SCH[SecurityContextHolder]
end

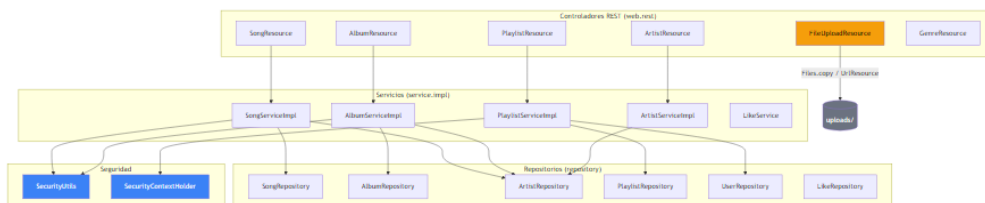
SR --> SS
AR --> AS
ATR --> ATS
PR --> PS

SS --> SRepo
SS --> ATRepo
SS --> SEC
AS --> ARepo
AS --> ATRepo
AS --> SEC
PS --> PRepo
PS --> URepo
PS --> SCH
ATS --> ATRepo

FU --> ["Files.copy / UrlResource" | DISK[(uploads/)]

style FU fill:#f59e0b,color:#000
style SEC fill:#3b82f6,color:#fff
style SCH fill:#3b82f6,color:#fff
style DISK fill:#6b7280,color:#fff

```



14. Diagrama de estados del reproductor Angular

stateDiagram-v2

[*] --> Detenido : App cargada

Detenido --> Reproduciendo : click Play / seleccionar canción

Reproduciendo --> Pausado : click Pause

Pausado --> Reproduciendo : click Play

Reproduciendo --> Detenido : click Stop / canción termina (no repeat)

Reproduciendo --> Reproduciendo : canción termina + repeat ON

Reproduciendo --> Silenciado : click Mute (isMuted = true)

Silenciado --> Reproduciendo : click Mute (isMuted = false)

Reproduciendo --> MostrandoLetras : click Botón Letras

MostrandoLetras --> Reproduciendo : click Botón Letras (toggle off)

state MostrandoLetras {

[*] --> CargandoLetras

```

    CargandoLetras --> LetrasDisponibles : API retorna letra
    CargandoLetras --> LetrasNoEncontradas : API retorna error / vacío
  }

```

```

note right of Reproduciendo
  isPlaying = true
  signal Angular activa
end note

```

```

note right of Pausado
  isPlaying = false
  audio.pause() llamado
end note

```

